

目录

文件上传漏洞修复报告 1

 目录 2

 1. 文件上传漏洞概述 2

 2. 漏洞一：无文件类型检查 2

 2.1 问题代码 2

 2.2 攻击示例 3

 2.3 危害等级 3

 3. 漏洞二：使用原始文件名保存 3

 3.1 问题代码 3

 3.2 风险分析 3

 3.3 修复方案 3

 4. 漏洞三：无文件内容校验 4

 4.1 问题描述 4

 4.2 修复方案：文件头魔数校验 4

 5. 漏洞四：文件名覆盖漏洞 4

 5.1 风险分析 4

 5.2 修复方案 5

 6. 漏洞五：路径穿越风险 5

 6.1 攻击示例 5

 6.2 修复方案 5

 7. 文件上传漏洞原理 5

 7.1 攻击流程 5

 7.2 为何服务器不应信任用户上传 6

 8. 修复方案 6

 8.1 安全上传函数完整代码 6

 9. 修复前后代码对比 7

 漏洞版（不做任何检查） 7

 安全版（三层检查） 8

 10. 安全建议 9

 10.1 文件上传安全规范 9

 10.2 文件上传防御金字塔 9

 10.3 开发者检查清单 10

 10.4 推荐：使用对象存储 10

 附录：OWASP 文件上传安全指南 10

文件上传漏洞修复报告

项目名称：用户信息管理系统
报告日期：2026-07-09
审计目标：Flask 用户管理平台中的文件上传漏洞分析与修复
报告人：安全审计组

目录

1. 文件上传漏洞概述
 2. 漏洞一：无文件类型检查
 3. 漏洞二：使用原始文件名保存
 4. 漏洞三：无文件内容校验
 5. 漏洞四：文件名覆盖漏洞
 6. 漏洞五：路径穿越风险
 7. 文件上传漏洞原理
 8. 修复方案
 9. 修复前后代码对比
 10. 安全建议
-

1. 文件上传漏洞概述

在本次安全审计中，发现目标系统的文件上传功能存在 **5 项安全漏洞**，全部由”不做任何检查”导致。

编号	漏洞名称	风险等级	攻击入口
FL-001	无文件类型检查	严重	上传任意后缀文件 (.exe/.php/.html)
FL-002	使用原始文件名	高危	文件名被直接用于保存和访问
FL-003	无文件内容校验	高危	伪装的恶意文件可通过后缀检查
FL-004	文件名覆盖	中危	同名文件直接覆盖
FL-005	路径穿越风险	中危	文件名包含../ 可逃逸上传目录

2. 漏洞一：无文件类型检查

2.1 问题代码

```
@app.route("/upload", methods=["GET", "POST"])
def upload():
    file = request.files.get("file")
    if file and file.filename:
        # 不做任何类型检查，直接保存
        filename = file.filename
        file.save(os.path.join("static/uploads", filename))
```

2.2 攻击示例

攻击者可上传：

文件名	用途	后果
shell.php	WebShell	如果在 PHP 环境下可直接执行命令
index.html	XSS 页面	包含恶意 JavaScript 脚本
virus.exe	恶意程序	供其他用户下载执行
malware.jar	Java 病毒	服务器端执行恶意代码

2.3 危害等级

攻击类型	可能性	影响
WebShell 上传	高	服务器被完全控制
XSS 攻击	高	用户信息被窃取
恶意文件分发	中	客户端被入侵

3. 漏洞二：使用原始文件名保存

3.1 问题代码

```
# 使用用户提供的原始文件名保存
filename = file.filename
file.save(os.path.join("static/uploads", filename))
```

3.2 风险分析

风险	说明
文件名预测	admin_avatar.png 等固定文件名可被猜到
文件名冲突	多用户上传同名文件会互相覆盖
特殊字符	文件名含中文或特殊字符可能导致路径解析错误
信息泄露	文件名可能包含敏感信息（如用户名、时间）

3.3 修复方案

```
import uuid
# 使用 UUID 重命名，彻底避免上述问题
```

```
ext = filename.rsplit(".", 1)[1].lower()
new_filename = f"{uuid.uuid4().hex}.{ext}"
```

4. 漏洞三：无文件内容校验

4.1 问题描述

只检查文件后缀是不够的，攻击者可以：

1. 将一个 shell.php 重命名为 shell.png 上传
2. 文件后缀是 .png，但实际内容是 PHP 代码
3. 如果服务器配置不当，依然可以以 PHP 执行

4.2 修复方案：文件头魔数校验

每种图片格式的文件头都有固定的字节序列（Magic Number / 魔数）：

```
IMAGE_SIGNATURES = {
    b"\xff\xd8\xff": "JPEG",
    b"\x89PNG": "PNG",
    b"GIF87a": "GIF",
    b"GIF89a": "GIF",
    b"RIFF": "WEBP",
    b"BM": "BMP",
}

def validate_image_header(file_path):
    """ 检查文件前几个字节是否匹配图片格式的魔数 """
    with open(file_path, "rb") as f:
        header = f.read(8)
        for sig in IMAGE_SIGNATURES:
            if header.startswith(sig):
                return True
        return False
```

这样即使攻击者把 shell.php 改成 shell.png，文件头魔数也不会欺骗过去。

5. 漏洞四：文件名覆盖漏洞

5.1 风险分析

如果用户 A 上传了 avatar.jpg，用户 B 也上传 avatar.jpg：

用户A上传 → static/uploads/avatar.jpg ← 正常
用户B上传 → static/uploads/avatar.jpg ← 覆盖了A的文件!

5.2 修复方案

使用 UUID 确保每次上传的文件名唯一:

```
import uuid

# 生成永不重复的文件名: a1b2c3d4e5f6...jpg
new_filename = f"{uuid.uuid4().hex}.{ext}"
```

6. 漏洞五: 路径穿越风险

6.1 攻击示例

攻击者上传文件名为:

../../etc/passwd

拼接后:

```
os.path.join("static/uploads", "../../etc/passwd")
# 实际路径: static/etc/passwd
# 文件被保存到 unexpected 位置!
```

6.2 修复方案

使用 UUID 重命名后, 文件名中不包含路径分隔符, 从根本上杜绝了路径穿越。

7. 文件上传漏洞原理

7.1 攻击流程

攻击者

服务器

上传 shell.php

→

提交恶意文件

检查? → 无

保存 static/uploads/shell.php

访问 http://target/shell.php →

访问上传的文件

执行恶意代码!

← 服务器权限被获取

服务器沦陷

7.2 为何服务器不应信任用户上传

因素	说明
文件名	用户可任意伪造，含特殊字符或路径
文件类型	HTTP Content-Type 可被篡改
文件内容	后缀名不代表真实内容
文件大小	超大文件可导致磁盘空间耗尽
文件数量	大量文件可导致 inode 耗尽

8. 修复方案

8.1 安全上传函数完整代码

```
import os
import uuid

# 允许的文件后缀列表（白名单）
ALLOWED_EXTENSIONS = {"png", "jpg", "jpeg", "gif", "webp", "bmp"}

# 常见图片文件头魔数
IMAGE_SIGNATURES = {
    b"\xff\xd8\xff": "jpg/jpeg",
    b"\x89PNG": "png",
    b"GIF87a": "gif",
    b"GIF89a": "gif",
    b"RIFF": "webp",
    b"BM": "bmp",
}

def allowed_file(filename):
    """ 白名单后缀检查 """
    return "." in filename and \
        filename.rsplit(".", 1)[1].lower() in ALLOWED_EXTENSIONS

def validate_image_header(file_path):
    """ 文件头魔数校验 """
    with open(file_path, "rb") as f:
        header = f.read(8)
    return any(header.startswith(sig) for sig in IMAGE_SIGNATURES)
```

```

@app.route("/upload", methods=["GET", "POST"])
def upload():
    if "username" not in session:
        return redirect("/login")

    if request.method == "POST":
        file = request.files.get("file")
        if not file or not file.filename:
            return render_template("upload.html", error=" 请选择一个文件")

        # 白名单后缀检查
        if not allowed_file(file.filename):
            return render_template("upload.html", error=" 不支持的文件类型")

        # UUID 重命名 (防覆盖 + 防路径穿越)
        ext = file.filename.rsplit(".", 1)[1].lower()
        new_filename = f"{uuid.uuid4().hex}.{ext}"
        save_path = os.path.join("static/uploads", new_filename)

        file.save(save_path)

        # 文件头魔数校验
        if not validate_image_header(save_path):
            os.remove(save_path)
            return render_template("upload.html", error=" 文件内容不是合法的图片格式")

        file_url = url_for("static", filename=f"uploads/{new_filename}")
        return render_template("upload.html", file_url=file_url, filename=new_filename)

    return render_template("upload.html")

```

9. 修复前后代码对比

漏洞版（不做任何检查）

```

@app.route("/upload", methods=["GET", "POST"])
def upload():
    if "username" not in session:
        return redirect("/login")

```

```

if request.method == "POST":
    file = request.files.get("file")
    if file and file.filename:
        filename = file.filename # 原始文件名, 可用 ../ 穿越
        file.save(os.path.join("static/uploads", filename)) # 不检查类型和内容
        file_url = url_for("static", filename=f"uploads/{filename}")
        return render_template("upload.html", file_url=file_url, filename=filename)
    else:
        return render_template("upload.html", error=" 请选择一个文件")

return render_template("upload.html")

```

安全版（三层检查）

```

ALLOWED_EXTENSIONS = {"png", "jpg", "jpeg", "gif", "webp", "bmp"}

IMAGE_SIGNATURES = {
    b"\xff\xd8\xff": "jpg/jpeg",
    b"\x89PNG": "png",
    b"GIF87a": "gif",
    b"GIF89a": "gif",
    b"RIFF": "webp",
    b"BM": "bmp",
}

def allowed_file(filename):
    return "." in filename and \
        filename.rsplit(".", 1)[1].lower() in ALLOWED_EXTENSIONS

def validate_image_header(file_path):
    with open(file_path, "rb") as f:
        header = f.read(8)
    return any(header.startswith(sig) for sig in IMAGE_SIGNATURES)

@app.route("/upload", methods=["GET", "POST"])
def upload():
    if "username" not in session:
        return redirect("/login")

    if request.method == "POST":
        file = request.files.get("file")
        if not file or not file.filename:
            return render_template("upload.html", error=" 请选择一个文件")

```



```

# 第一层：白名单后缀检查
if not allowed_file(file.filename):
    return render_template("upload.html", error=" 不支持的文件类型，仅允许图片文件")

# 第二层：UUID 重命名（防覆盖、防路径穿越）
ext = file.filename.rsplit(".", 1)[1].lower()
new_filename = f"{uuid.uuid4().hex}.{ext}"
save_path = os.path.join("static/uploads", new_filename)

file.save(save_path)

# 第三层：文件头魔数校验
if not validate_image_header(save_path):
    os.remove(save_path)
    return render_template("upload.html", error=" 文件内容不是合法的图片格式")

file_url = url_for("static", filename=f"uploads/{new_filename}")
return render_template("upload.html", file_url=file_url, filename=new_filename)

return render_template("upload.html")

```

10. 安全建议

10.1 文件上传安全规范

检查项	说明	优先级
① 白名单后缀	只允许必要的文件类型	必须
② UUID 重命名	使用随机字符串，不暴露原始文件名	必须
③ 文件头校验	检查魔数，确认文件类型真实	必须
④ 文件大小限制	设置 MAX_CONTENT_LENGTH	必须
⑤ 存储目录权限	设置为不可执行脚本	推荐
⑥ CDN/对象存储	上传到 OSS/S3，与应用服务器分离	推荐

10.2 文件上传防御金字塔

- 白名单后缀 ← 第一道防线，拦截大部分恶意文件
- UUID 重命名 ← 防覆盖、防路径穿越

文件头校验	← 防伪装后缀
大小限制	← 防磁盘耗尽
不可执行目录	← 防 WebShell 执行

10.3 开发者检查清单

- ☐ 使用白名单限制上传文件的后缀
- ☐ 使用 UUID 或时间戳重命名文件
- ☐ 检查文件头魔数验证内容
- ☐ 限制上传文件的最大体积
- ☐ 文件存储到不可执行脚本的目录
- ☐ 返回的 URL 使用随机化路径
- ☐ 定期清理未使用的上传文件
- ☐ 不要在文件名中包含用户输入

10.4 推荐：使用对象存储

对于生产环境，建议将文件上传到对象存储服务（如阿里云 OSS、AWS S3）而不是本地服务器：

用户上传 → 应用服务器校验 → 上传到 OSS → 返回 OSS URL

优点：

- 存储与应用分离，WebShell 无法执行
- 自带 CDN 加速
- 独立权限管理
- 弹性扩展

附录：OWASP 文件上传安全指南

OWASP 关于文件上传的推荐做法：

1. **使用白名单验证**：只允许特定文件类型，不要用黑名单
 2. **验证文件内容**：使用文件头魔数，不要只依赖 Content-Type
 3. **重命名文件**：使用随机化文件名，不暴露原始文件名
 4. **限制文件大小**：设置合理的最大文件体积
 5. **限制存储权限**：上传目录不应具有执行权限
 6. **使用专用存储**：将上传文件与应用代码分离部署
-

报告编写: Claude Code AI Assistant
报告版本: v1.0
参考标准: OWASP Top 10 (2021) - A03:2021 Injection
测试环境: Flask 用户管理系统
免责声明: 本报告仅用于安全学习和教学目的